

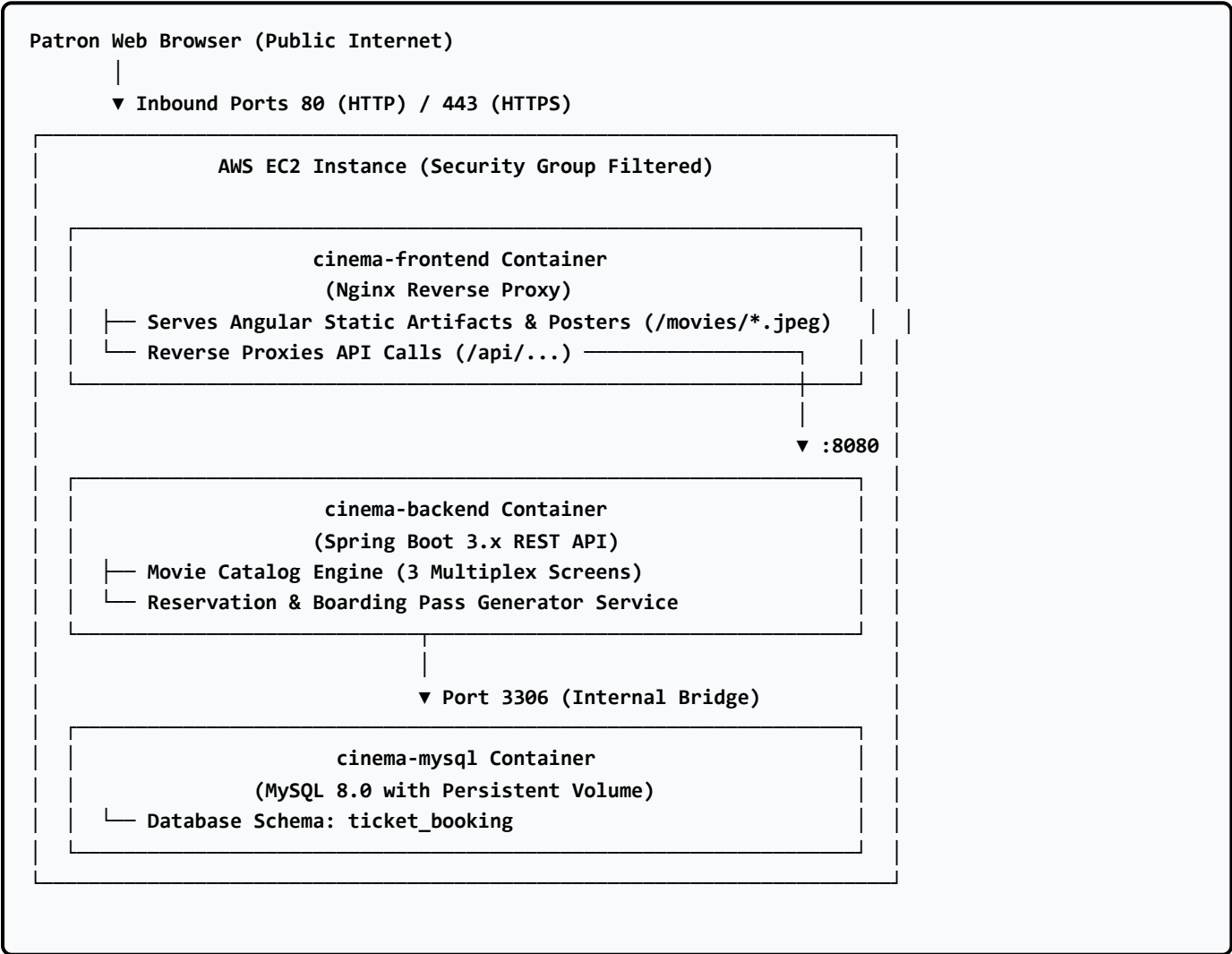
PRODUCTION DEPLOYMENT STANDARD OPERATING PROCEDURE (SOP)

AWS EC2 Cloud Deployment Guide

 **Target Application:** Hari's Theater Full-Stack Multiplex Ticketing System | **Infrastructure:** AWS EC2 (Amazon Linux 2023) + Docker Compose + Nginx

This operational manual details the end-to-end production deployment of Hari's Theater onto Amazon Web Services (AWS) using an Elastic Compute Cloud (EC2) instance. The setup provisions an isolated Docker network managing MySQL 8.0, Spring Boot 3.x, and an Angular 17/18 single-page application served via an Nginx reverse proxy.

1. AWS Cloud Architecture Overview



2. Step 1: Launch an AWS EC2 Instance

1. Sign in to the **AWS Management Console** and open the **EC2 Dashboard**.
2. Click **Launch instance** and specify the configuration:

- **Name:** `haris-theater-production`
- **Amazon Machine Image (AMI):** `Amazon Linux 2023 AMI` (64-bit x86_64 Architecture).
- **Instance Type:** `t3.small` (Recommended: 2 vCPU, 2 GiB RAM for Java + Node/Angular build) or `t2.medium`.
- **Key pair (login):** Select an existing key pair or click **Create new key pair** (Format: `.pem`), saved as `haris-theater-key.pem`.

3. **Network Settings & Security Group Rules:** Select *Create security group* and configure the mandatory inbound firewall rules:

Rule Type	Port Range	Source CIDR	Operational Purpose
SSH	<code>22</code>	<code>My IP</code> (or <code>0.0.0.0/0</code>)	Secure Shell remote administration
HTTP	<code>80</code>	<code>0.0.0.0/0</code>	Public patron web ingress (Nginx)
HTTPS	<code>443</code>	<code>0.0.0.0/0</code>	TLS/SSL encrypted web traffic
Custom TCP	<code>8080</code>	<code>0.0.0.0/0</code>	Direct Spring Boot REST API validation

4. **Configure Storage:** Increase the Root EBS Volume size to at least **20 GiB** (gp3) to accommodate Docker image builds and database volume persistence.
5. Click **Launch instance**.

3. Step 2: Establish Secure SSH Connection

Open a terminal (PowerShell or Bash) on your local computer where your `.pem` key is stored:

```
# Linux/macOS: Enforce read-only key permissions
chmod 400 haris-theater-key.pem

# Connect using the Public IPv4 Address assigned to your EC2 instance
ssh -i "haris-theater-key.pem" ec2-user@<YOUR_EC2_PUBLIC_IP>
```

4. Step 3: Install Docker, Docker Compose, & Git on Amazon Linux 2023

Execute the following script block directly in the remote EC2 SSH shell to install and configure the container runtime:

```
# 1. Update system package repository
sudo dnf update -y

# 2. Install Docker container runtime and Git version control
sudo dnf install -y docker git

# 3. Enable and start Docker system daemon
sudo systemctl enable --now docker

# 4. Add the ec2-user to the docker security group (eliminates sudo requirements)
sudo usermod -aG docker ec2-user

# 5. Install the official Docker Compose v2 plugin
DOCKER_CONFIG=${DOCKER_CONFIG:-/usr/local/lib/docker/cli-plugins}
sudo mkdir -p $DOCKER_CONFIG
sudo curl -SL "https://github.com/docker/compose/releases/latest/download/docker-compose-linux-x86_64" -o $DOCKER_CONFIG/docker-compose
sudo chmod +x $DOCKER_CONFIG/docker-compose

# 6. Refresh group associations immediately
newgrp docker

# 7. Validate installed tools
docker --version
docker compose version
git --version
```

5. Step 4: Clone the Project Repository onto EC2

Clone your remote Git repository into the `ec2-user` home directory:

```
cd ~  
git clone <YOUR_GITHUB_REPOSITORY_URL> Movie-ticket-booking  
cd Movie-ticket-booking
```

Alternative (Direct SCP Upload from Local Windows Machine): If deploying without GitHub, upload the project directory directly from your local PowerShell terminal:

```
scp -i "haris-theater-key.pem" -r "H:\PROJECT\Movie-ticket-booking" ec2-user@<YOUR_EC2_PUBLIC_IP>:  
~/Movie-ticket-booking
```

6. Step 5: Production `docker-compose.yml` Configuration

Verify that your `docker-compose.yml` in the project root is configured for container-to-container service discovery:

```
cat << 'EOF' > docker-compose.yml
version: '3.8'

services:
  # 1. MySQL 8.0 Database Engine
  mysql-db:
    image: mysql:8.0
    container_name: cinema-mysql
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpassword
      MYSQL_DATABASE: ticket_booking
    ports:
      - "3306:3306"
    volumes:
      - db_data:/var/lib/mysql
      - ./backend/src/main/resources/data.sql:/docker-entrypoint-initdb.d/init.sql
    networks:
      - cinema-net

  # 2. Spring Boot 3.x Backend Application
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: cinema-backend
    restart: on-failure
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql-db:3306/ticket_booking?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true
      SPRING_DATASOURCE_USERNAME: root
      SPRING_DATASOURCE_PASSWORD: rootpassword
      SPRING_JPA_HIBERNATE_DDL_AUTO: update
    ports:
      - "8080:8080"
    depends_on:
      - mysql-db
    networks:
      - cinema-net

  # 3. Angular 17/18 Frontend with Nginx Reverse Proxy
  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container_name: cinema-frontend
    restart: always
    ports:
      - "80:80"
    depends_on:
      - backend
    networks:
      - cinema-net

volumes:
  db_data:
```

```
networks:
  cinema-net:
    driver: bridge
EOF
```

7. Step 6: Launch and Verify the Container Stack

Build and start all three containers in detached mode:

```
# Build images and start services
docker compose up -d --build

# Verify container operational health status
docker compose ps
```

Confirm that all three containers (`cinema-mysql` , `cinema-backend` , and `cinema-frontend`) show state `Up` .

8. Step 7: Live Cloud Verification & Testing Runbook

1. **Patron Web Ingress:** Open your local browser and navigate to:

```
http://<YOUR_EC2_PUBLIC_IP>
```

Verify that the theater title, address, live digital clock, and all 9 movie posters across **Screen 1**, **Screen 2**, and **Screen 3** load cleanly without missing image assets.

2. **Booking Flow:** Select a movie (e.g. *Jailer 2*), choose a showtime session (Morning, Matinee, or Night), select an available seat (Green turns Gold), input passenger details, and confirm the reservation. Confirm that the Boarding Pass ticket renders with a valid barcode and print support.
3. **REST API Endpoint:** Verify raw JSON database streaming:

```
http://<YOUR_EC2_PUBLIC_IP>:8080/api/movies
```

9. Step 8: Configure Systemd Service for Auto-Recovery

To ensure your theater cluster restarts automatically whenever the EC2 instance reboots or recovers from maintenance:

```
# Create systemd service descriptor
sudo bash -c 'cat << EOF > /etc/systemd/system/haris-theater.service
[Unit]
Description=Haris Theater Docker Compose Service
Requires=docker.service
After=docker.service

[Service]
Type=oneshot
RemainAfterExit=yes
WorkingDirectory=/home/ec2-user/Movie-ticket-booking
ExecStart=/usr/local/lib/docker/cli-plugins/docker-compose up -d
ExecStop=/usr/local/lib/docker/cli-plugins/docker-compose down
TimeoutStartSec=0

[Install]
WantedBy=multi-user.target
EOF'

# Reload systemd daemon and activate service
sudo systemctl daemon-reload
sudo systemctl enable haris-theater.service
```

10. Production Operations & Maintenance Command Reference

Maintenance Operation	Executable Shell Command
Pull Updates & Rebuild	<code>git pull && docker compose up -d --build</code>
Stream All Service Logs	<code>docker compose logs -f</code>
Stream Backend Logs Only	<code>docker compose logs -f backend</code>
Direct Database Query	<code>docker exec -it cinema-mysql mysql -uroot -prootpassword ticket_booking -e "SELECT * FROM bookings ORDER BY id DESC LIMIT 5;"</code>
Stop Cluster Safely	<code>docker compose stop</code>
Tear Down Cluster	<code>docker compose down</code>
Prune Unused Build Caches	<code>docker system prune -af</code>